

[pd] Lecture de fichiers – read_*

DESCRIPTION : Charge des données depuis des fichiers ou sources externes en DataFrame

POURQUOI : Point d'entrée principal de pandas. Supporte des dizaines de formats.

CONTEXTE : Import de données, ETL, pipelines de traitement.

FONCTIONS PRINCIPALES :

<code>pd.read_csv("f.csv")</code>	Lit un fichier CSV
<code>pd.read_csv("f.csv", sep=";")</code>	Séparateur personnalisé
<code>pd.read_csv("f.csv", header=None)</code>	Sans ligne d'en-tête
<code>pd.read_csv("f.csv", names=["a","b"])</code>	Noms de colonnes personnalisés
<code>pd.read_csv("f.csv", index_col=0)</code>	Première colonne comme index
<code>pd.read_csv("f.csv", usecols=["a","b"])</code>	Colonnes spécifiques seulement
<code>pd.read_csv("f.csv", dtype={"col": int})</code>	Types forcés
<code>pd.read_csv("f.csv", nrows=100)</code>	Lit seulement N lignes
<code>pd.read_csv("f.csv", skiprows=5)</code>	Saute les N premières lignes
<code>pd.read_csv("f.csv", chunksize=1000)</code>	Lit par chunks (gros fichiers)
<code>pd.read_csv("f.csv", compression="gzip")</code>	Lit un fichier compressé
<code>pd.read_csv("f.csv", encoding="utf-8")</code>	Encodage spécifique
<code>pd.read_csv("f.csv", na_values=["N/A","-"])</code>	Valeurs considérées comme NaN
<code>pd.read_csv("f.csv", parse_dates=["date"])</code>	Parse une colonne en datetime
<code>pd.read_csv("f.csv", on_bad_lines="skip")</code>	Ignore les lignes malformées
<code>pd.read_excel("f.xlsx")</code>	Lit un fichier Excel
<code>pd.read_excel("f.xlsx", sheet_name="Sheet1")</code>	Feuille spécifique
<code>pd.read_excel("f.xlsx", sheet_name=None)</code>	Toutes les feuilles (dict)
<code>pd.read_json("f.json")</code>	Lit un fichier JSON
<code>pd.read_json("f.json", orient="records")</code>	Orientation spécifique
<code>pd.read_parquet("f.parquet")</code>	Lit un fichier Parquet
<code>pd.read_parquet("f.parquet", columns=["a"])</code>	Colonnes spécifiques
<code>pd.read_feather("f.feather")</code>	Lit un fichier Feather
<code>pd.read_hdf("f.h5", key="data")</code>	Lit un fichier HDF5
<code>pd.read_pickle("f.pkl")</code>	Lit un fichier Pickle
<code>pd.read_sql("SELECT * FROM t", conn)</code>	Lit depuis une base SQL
<code>pd.read_sql_query("SELECT ...", conn)</code>	Requête SQL explicite
<code>pd.read_sql_table("table", conn)</code>	Lit une table entière
<code>pd.read_html("url")</code>	Lit les tableaux d'une page HTML
<code>pd.read_xml("f.xml")</code>	Lit un fichier XML
<code>pd.read_fwf("f.txt")</code>	Lit un fichier à largeur fixe
<code>pd.read_table("f.tsv")</code>	Lit un fichier délimité
<code>pd.read_clipboard()</code>	Lit depuis le presse-papiers
<code>pd.read_stata("f.dta")</code>	Lit un fichier Stata
<code>pd.read_sas("f.sas7bdat")</code>	Lit un fichier SAS
<code>pd.read_spss("f.sav")</code>	Lit un fichier SPSS
<code>pd.read_orc("f.orc")</code>	Lit un fichier ORC

EXEMPLES :

```
df = pd.read_csv("data.csv", sep=";", encoding="utf-8", parse_dates=["date"])
df = pd.read_csv("gros.csv", chunksize=10000) # retourne un itérateur
for chunk in df: print(chunk.shape)
df = pd.read_excel("rapport.xlsx", sheet_name="Ventes", usecols="A:D")
df = pd.read_sql("SELECT * FROM clients WHERE actif=1", connexion)
```

[df] Écriture de fichiers – to_*

DESCRIPTION : Exporte un DataFrame vers différents formats de fichiers

POURQUOI : Sauvegarde des résultats d'analyse, échange de données entre systèmes.

CONTEXTE : Export de rapports, pipelines de données, sauvegardes.

FONCTIONS PRINCIPALES :

<code>df.to_csv("f.csv", index=False)</code>	Exporte en CSV
<code>df.to_csv("f.csv", sep=";")</code>	Séparateur personnalisé
<code>df.to_csv("f.csv", encoding="utf-8")</code>	Encodage spécifique
<code>df.to_csv("f.csv", compression="gzip")</code>	Exporte compressé
<code>df.to_csv("f.csv", mode="a", header=False)</code>	Ajoute à un fichier existant
<code>df.to_csv("f.csv", columns=["a","b"])</code>	Colonnes spécifiques
<code>df.to_csv("f.csv", float_format="%.2f")</code>	Format des flottants
<code>df.to_excel("f.xlsx", index=False)</code>	Exporte en Excel
<code>df.to_excel("f.xlsx", sheet_name="Données")</code>	Nom de feuille
<code>df.to_excel(writer, sheet_name="S1")</code>	Via ExcelWriter (multi-feuilles)
<code>df.to_json("f.json")</code>	Exporte en JSON
<code>df.to_json("f.json", orient="records")</code>	Format liste d'objets

```

df.to_json("f.json", indent=2)           JSON indenté lisible
df.to_parquet("f.parquet", index=False)   Exporte en Parquet
df.to_parquet("f.parquet", compression="snappy") Compression snappy
df.to_feather("f.feather")               Exporte en Feather
df.to_hdf("f.h5", key="data")            Exporte en HDF5
df.to_pickle("f.pkl")                    Exporte en Pickle
df.to_sql("table", conn, if_exists="replace") Exporte en base SQL
df.to_sql("table", conn, if_exists="append") Ajoute en base SQL
df.to_html("f.html")                     Exporte en HTML
df.to_xml("f.xml")                        Exporte en XML
df.to_markdown()                          Retourne une chaîne Markdown
df.to_latex()                             Retourne une chaîne LaTeX
df.to_clipboard()                         Copie dans le presse-papiers
df.to_string()                            Retourne une chaîne lisible
df.to_dict()                             Convertit en dictionnaire Python
df.to_records()                           Convertit en tableau numpy
df.to_numpy()                             Retourne un array numpy

```

EXEMPLES :

```

df.to_csv("résultat.csv", index=False, sep=";", encoding="utf-8")
df.to_parquet("archive.parquet", compression="zstd", index=False)
with pd.ExcelWriter("rapport.xlsx") as w:
    df1.to_excel(w, sheet_name="Ventes")
    df2.to_excel(w, sheet_name="Clients")

```

[df] Exploration – info, describe, head, tail, shape...

DESCRIPTION : Inspecte la structure, les types et le contenu d'un DataFrame
 POURQUOI : Première étape de toute analyse de données.
 CONTEXTE : EDA (Exploratory Data Analysis), diagnostic, audit de données.
 FONCTIONS PRINCIPALES :

```

df.head(n=5)           N premières lignes
df.tail(n=5)           N dernières lignes
df.sample(n=10)        N lignes aléatoires
df.sample(frac=0.1)    Fraction aléatoire (10%)
df.shape               (nb_lignes, nb_colonnes)
df.size               Nombre total de cellules
df.ndim               Nombre de dimensions
df.dtypes             Types de chaque colonne
df.columns            Liste des colonnes
df.index              Index du DataFrame
df.axes               [index, colonnes]
df.values             Données brutes (array numpy)
df.info()             Résumé : types, nulls, mémoire
df.info(memory_usage="deep") Taille mémoire précise
df.describe()         Statistiques descriptives (num)
df.describe(include="all") Inclut colonnes catégorielles
df.describe(include="object") Colonnes texte seulement
df.memory_usage(deep=True) Mémoire par colonne en octets
df.memory_usage(deep=True).sum() Mémoire totale
df.count()            Nombre de valeurs non-nulles
df.nunique()          Nombre de valeurs uniques/colonne
df["col"].unique()    Valeurs uniques d'une colonne
df["col"].value_counts() Fréquences des valeurs
df["col"].value_counts(normalize=True) Fréquences relatives (%)
df.empty              True si le DataFrame est vide

```

EXEMPLES :

```

print(df.shape, df.dtypes)
df.info(memory_usage="deep")
df["catégorie"].value_counts().head(10)

```

[df] Création et construction

DESCRIPTION : Crée des DataFrames et Series depuis diverses sources
 POURQUOI : Construire des données de test, transformer des structures existantes.
 CONTEXTE : Tests unitaires, données synthétiques, conversion de structures Python.
 FONCTIONS PRINCIPALES :

```

pd.DataFrame({"a": [1,2], "b": [3,4]})      Depuis un dictionnaire
pd.DataFrame([(1,2),(3,4)], columns=["a","b"]) Depuis une liste de tuples
pd.DataFrame(array, columns=["a","b"])      Depuis un array numpy
pd.DataFrame.from_dict(d)                   Depuis un dictionnaire
pd.DataFrame.from_records(liste)            Depuis une liste de dicts
pd.Series([1, 2, 3])                       Crée une Series
pd.Series({"a": 1, "b": 2})                 Series depuis un dict

```

```

pd.RangeIndex(start=0, stop=10, step=1)      Index numérique
pd.Index(["a", "b", "c"])                    Index personnalisé
pd.MultiIndex.from_tuples([("a",1),("b",2)]) Index multi-niveaux
pd.date_range("2024-01-01", periods=365)      Index de dates
pd.date_range("2024-01-01", "2024-12-31", freq="D") Plage de dates
pd.timedelta_range("0 days", periods=10)      Index de durées
pd.period_range("2024-01", periods=12, freq="M") Index de périodes
pd.interval_range(start=0, end=10, periods=5) Index d'intervalles
pd.bdate_range("2024-01-01", periods=20)      Jours ouvrables

```

EXEMPLES :

```

df = pd.DataFrame({"nom": ["Alice","Bob"], "age": [30, 25], "ville": ["Paris","Lyon"]})
dates = pd.date_range("2024-01-01", periods=100, freq="D")
df = pd.DataFrame({"valeur": range(100)}, index=dates)

```

[df] Sélection et indexation – loc, iloc, at, iat

DESCRIPTION : Accède et sélectionne des sous-ensembles du DataFrame
 POURQUOI : Extraire des lignes, colonnes, cellules spécifiques.
 CONTEXTE : Filtrage, extraction, mise à jour ciblée.

FONCTIONS PRINCIPALES :

df["col"]	Sélectionne une colonne (Series)
df[["col1","col2"]]	Sélectionne plusieurs colonnes
df.loc[label]	Ligne par label d'index
df.loc["a":"c"]	Plage de labels
df.loc[:, "col"]	Toutes lignes, une colonne
df.loc[df["col"] > 5]	Sélection par condition
df.loc[condition, "col"]	Ligne+colonne par condition
df.iloc[0]	Première ligne (par position)
df.iloc[0:5]	5 premières lignes
df.iloc[:, 0]	Première colonne
df.iloc[0:5, 1:3]	Sous-matrice par position
df.at[label, "col"]	Valeur unique par label (rapide)
df.iat[0, 1]	Valeur unique par position (rapide)
df.get("col", default=None)	Colonne avec valeur par défaut
df.xs("label", level=0)	Coupe transversale (MultiIndex)
df.take([0, 2, 4])	Lignes par position (liste)
pd.IndexSlice[:]	Raccourci pour slicing MultiIndex

EXEMPLES :

```

df.loc[df["age"] > 30, ["nom", "ville"]]
df.iloc[0:100, [0, 2, 5]]
df.at[0, "nom"] = "Charlie"

```

[df] Filtrage et requêtes – query, filter, where, mask, isin

DESCRIPTION : Filtre les lignes et colonnes selon des conditions
 POURQUOI : Sélectionner des sous-ensembles de données pertinents.
 CONTEXTE : Analyse exploratoire, nettoyage, segmentation.

FONCTIONS PRINCIPALES :

df[df["col"] > 10]	Filtre par condition simple
df[(df["a"] > 5) & (df["b"] == "X")]	Conditions multiples (ET)
df[(df["a"] > 5) (df["b"] == "X")]	Conditions multiples (OU)
df[~(df["col"] > 10)]	Négation d'une condition
df.query("col > 10")	Filtre par expression string
df.query("a > @seuil")	Utilise une variable locale
df.query("col in @liste")	Filtre avec une liste
df["col"].isin([1, 2, 3])	Masque : valeurs dans une liste
df[df["col"].isin([1, 2, 3])]	Filtre avec isin
df[~df["col"].isin([1, 2, 3])]	Exclusion avec isin
df.filter(items=["col1","col2"])	Filtre les colonnes par nom
df.filter(like="prix")	Colonnes contenant "prix"
df.filter(regex="^col_")	Colonnes par regex
df.where(condition)	Garde les vrais, NaN ailleurs
df.mask(condition)	Remplace les vrais par NaN
df["col"].between(a, b)	Valeurs entre a et b inclus
df.truncate(before=5, after=10)	Tronque par index
df.asof(index)	Dernière valeur connue avant index

EXEMPLES :

```

df.query("age >= 18 and ville == 'Paris'")
df[df["catégorie"].isin(["A", "B", "C"])]
df.filter(regex="^montant_")

```

[df] Modification et transformation

DESCRIPTION	: Modifie la structure ou les valeurs d'un DataFrame
POURQUOI	: Préparer, nettoyer et enrichir les données.
CONTEXTE	: Feature engineering, nettoyage, transformation de colonnes.
FONCTIONS PRINCIPALES :	
df["col"] = valeur	Ajoute ou modifie une colonne
df.assign(col=valeur)	Ajoute une colonne (retourne copie)
df.assign(col=lambda x: x["a"] + x["b"])	Colonne calculée
df.insert(pos, "col", valeur)	Insère une colonne à la position
df.pop("col")	Supprime et retourne une colonne
df.drop("col", axis=1)	Supprime une colonne
df.drop(["c1", "c2"], axis=1)	Supprime plusieurs colonnes
df.drop(index=0)	Supprime une ligne
df.drop(index=[0,1,2])	Supprime plusieurs lignes
df.rename(columns={"ancien": "nouveau"})	Renomme des colonnes
df.rename(index=0: "a")	Renomme des lignes
df.rename_axis("nom_index")	Renomme l'axe d'index
df.set_index("col")	Définit une colonne comme index
df.reset_index()	Réinitialise l'index (en colonne)
df.reset_index(drop=True)	Réinitialise sans garder l'ancien
df.set_axis(["a", "b"], axis=1)	Redéfinit les noms d'axes
df.reindex(nouvel_index)	Réindexe (ajoute NaN si manquant)
df.reindex_like(autre_df)	Réindexe comme un autre DataFrame
df.copy()	Copie complète (deep copy)
df.copy(deep=False)	Copie superficielle
df.astype({"col": float})	Change le type d'une colonne
df.astype(str)	Tout convertir en string
df.convert_dtypes()	Inférence automatique des types
df.infer_objects()	Inférence des types objets
df.add_prefix("préfixe_")	Ajoute un préfixe aux colonnes
df.add_suffix("_suffixe")	Ajoute un suffixe aux colonnes
df.pipe(fonction)	Applique une fonction au DataFrame
df.eval("c = a + b")	Évalue une expression
df.isetitem(pos, valeur)	Modifie une colonne par position
df.update(autre_df)	Met à jour avec un autre DataFrame
df.flags	Accès aux drapeaux internes
df.set_flags(allows_duplicate_labels=False)	Interdit les labels dupliqués
EXEMPLES :	
df = df.assign(total=df["qté"] * df["prix"], tva=lambda x: x["total"] * 0.2)	
df.rename(columns=str.lower).reset_index(drop=True)	

[df] Nettoyage – dropna, fillna, replace, duplicates

DESCRIPTION	: Gère les valeurs manquantes, doublons et données invalides
POURQUOI	: Les données réelles sont rarement propres. Étape cruciale avant analyse.
CONTEXTE	: Préparation de données, qualité des données, data cleaning.
FONCTIONS PRINCIPALES :	
df.isnull()	Masque des valeurs nulles
df.isna()	Identique à isnull()
df.notnull()	Masque des valeurs non-nulles
df.notna()	Identique à notnull()
df.isnull().sum()	Nombre de nulls par colonne
df.isnull().sum().sum()	Nombre total de nulls
df.dropna()	Supprime les lignes avec NaN
df.dropna(axis=1)	Supprime les colonnes avec NaN
df.dropna(how="all")	Supprime seulement si tout NaN
df.dropna(subset=["col"])	Supprime si NaN dans col spécifique
df.dropna(thresh=N)	Garde si au moins N valeurs non-null
df.fillna(0)	Remplace NaN par 0
df.fillna(method="ffill")	Forward fill (propagation avant)
df.ffill()	Raccourci forward fill
df.bfill()	Backward fill (propagation arrière)
df.fillna(df.mean())	Remplace par la moyenne
df.fillna({"col1": 0, "col2": "inconnu"})	Valeurs différentes par colonne
df.interpolate()	Interpolation linéaire
df.interpolate(method="time")	Interpolation temporelle
df.replace(ancien, nouveau)	Remplace une valeur
df.replace([1,2], [10,20])	Remplace plusieurs valeurs
df.replace({"col": {1: 10}})	Remplace dans une colonne
df.replace(regex=True, ...)	Remplacement par regex
df.duplicated()	Masque des lignes dupliquées
df.duplicated(subset=["col"])	Dupliqués sur une colonne
df.duplicated(keep="last")	Garde la dernière occurrence
df.drop_duplicates()	Supprime les doublons

<code>df.drop_duplicates(subset=["col"])</code>	Déduplique sur une colonne
<code>df.drop_duplicates(keep=False)</code>	Supprime toutes les occurrences
<code>pd.isna(valeur)</code>	Teste si une valeur est NaN
<code>pd.notna(valeur)</code>	Teste si une valeur n'est pas NaN

EXEMPLES :

```
df = df.dropna(subset=["id", "date"]).fillna({"montant": 0, "statut": "inconnu"})
df = df.drop_duplicates(subset=["id"], keep="last")
```

[df] Opérations sur les chaînes – str.*

DESCRIPTION : Applique des opérations de chaîne à une colonne de type texte
POURQUOI : Nettoyage et transformation de données textuelles.

CONTEXTE : Normalisation de données, extraction d'informations, parsing.

FONCTIONS PRINCIPALES :

<code>df["col"].str.lower()</code>	Minuscules
<code>df["col"].str.upper()</code>	Majuscules
<code>df["col"].str.strip()</code>	Supprime espaces début/fin
<code>df["col"].str.lstrip()</code>	Supprime espaces à gauche
<code>df["col"].str.rstrip()</code>	Supprime espaces à droite
<code>df["col"].str.replace("a","b")</code>	Remplace une sous-chaîne
<code>df["col"].str.replace(regex=True, ...)</code>	Remplacement par regex
<code>df["col"].str.contains("motif")</code>	Test d'inclusion (masque bool)
<code>df["col"].str.contains("motif", na=False)</code>	Gère les NaN
<code>df["col"].str.startswith("préfixe")</code>	Commence par
<code>df["col"].str.endswith("suffixe")</code>	Finit par
<code>df["col"].str.match("^regex\$")</code>	Correspond à un regex
<code>df["col"].str.fullmatch("^regex\$")</code>	Correspondance totale
<code>df["col"].str.extract(r"(\d+)")</code>	Extrait un groupe regex
<code>df["col"].str.extractall(r"(\d+)")</code>	Extrait tous les groupes
<code>df["col"].str.findall(r"\d+")</code>	Trouve toutes les occurrences
<code>df["col"].str.split(",")</code>	Divise en liste
<code>df["col"].str.split(",", expand=True)</code>	Divise en colonnes
<code>df["col"].str.join(sep)</code>	Joint une liste en chaîne
<code>df["col"].str.cat(sep=" ")</code>	Concatène toutes les valeurs
<code>df["col"].str.len()</code>	Longueur de chaque chaîne
<code>df["col"].str.count("motif")</code>	Compte les occurrences
<code>df["col"].str.get(i)</code>	i-ème caractère
<code>df["col"].str.slice(0, 5)</code>	Sous-chaîne [0:5]
<code>df["col"].str.pad(width=10, fillchar=" ")</code>	Aligne sur N caractères
<code>df["col"].str.zfill(5)</code>	Complète avec des zéros
<code>df["col"].str.title()</code>	Première lettre majuscule
<code>df["col"].str.capitalize()</code>	Première lettre maj, reste min
<code>df["col"].str.isdigit()</code>	Teste si numérique
<code>df["col"].str.isalpha()</code>	Teste si alphabétique
<code>df["col"].str.isnumeric()</code>	Teste si numérique (unicode)
<code>df["col"].str.encode("utf-8")</code>	Encode en bytes
<code>df["col"].str.decode("utf-8")</code>	Décode des bytes
<code>df["col"].str.normalize("NFC")</code>	Normalisation unicode
<code>df["col"].str.translate(table)</code>	Traduction de caractères
<code>df["col"].str.wrap(width=80)</code>	Retour à la ligne automatique

EXEMPLES :

```
df["nom"] = df["nom"].str.strip().str.lower()
df[df["email"].str.contains(r"@w+\.w+", regex=True, na=False)]
df[["prénom", "nom"]] = df["nom_complet"].str.split(" ", expand=True)
```

[df] Dates et temps – dt.* / to_datetime / date_range

DESCRIPTION : Manipule les données temporelles et les séries chronologiques
POURQUOI : Les données temporelles nécessitent un traitement spécifique.

CONTEXTE : Séries temporelles, analyses par période, resampling.

FONCTIONS PRINCIPALES :

<code>pd.to_datetime(df["col"])</code>	Convertit en datetime
<code>pd.to_datetime(df["col"], format="%d/%m/%Y")</code>	Format personnalisé
<code>pd.to_datetime(df["col"], errors="coerce")</code>	NaT si invalide
<code>pd.to_timedelta(df["col"])</code>	Convertit en timedelta
<code>pd.Timestamp("2024-01-01")</code>	Crée un timestamp
<code>pd.NaT</code>	Valeur manquante temporelle
<code>df["col"].dt.year</code>	Année
<code>df["col"].dt.month</code>	Mois
<code>df["col"].dt.day</code>	Jour
<code>df["col"].dt.hour</code>	Heure
<code>df["col"].dt.minute</code>	Minute
<code>df["col"].dt.second</code>	Seconde

df["col"].dt.dayofweek	Jour de la semaine (0=lundi)
df["col"].dt.day_name()	Nom du jour ("Monday"...)
df["col"].dt.month_name()	Nom du mois ("January"...)
df["col"].dt.dayofyear	Jour de l'année (1-365)
df["col"].dt.weekofyear	Semaine de l'année
df["col"].dt.quarter	Trimestre (1-4)
df["col"].dt.is_month_start	Vrai si début de mois
df["col"].dt.is_month_end	Vrai si fin de mois
df["col"].dt.is_year_start	Vrai si début d'année
df["col"].dt.is_year_end	Vrai si fin d'année
df["col"].dt.date	Date seule (sans heure)
df["col"].dt.time	Heure seule (sans date)
df["col"].dt.normalize()	Réinitialise l'heure à 00:00
df["col"].dt.floor("H")	Arrondi à l'heure inférieure
df["col"].dt.ceil("H")	Arrondi à l'heure supérieure
df["col"].dt.round("H")	Arrondi à l'heure la plus proche
df["col"].dt.tz_localize("UTC")	Applique un fuseau horaire
df["col"].dt.tz_convert("Europe/Paris")	Convertit le fuseau horaire
df["col"].dt.strftime("%d/%m/%Y")	Formate en chaîne
df["col"].dt.total_seconds()	Durée totale en secondes
df["col"].dt.components	Décompose un timedelta
df.resample("M").sum()	Agrège par mois
df.resample("W").mean()	Agrège par semaine
df.asfreq("D")	Change la fréquence
df.at_time("09:00")	Filtre par heure exacte
df.between_time("09:00", "17:00")	Filtre par plage horaire
df.shift(1)	Décale d'une période
df.shift(1, freq="D")	Décale d'un jour
df.diff()	Différence avec la période précédente
df.pct_change()	Variation en pourcentage
df.tshift(periods=1)	Décale l'index temporel

EXEMPLES :

```
df["date"] = pd.to_datetime(df["date"], format="%d/%m/%Y")
df["année"] = df["date"].dt.year
df.set_index("date").resample("M").agg({"ventes": "sum", "clients": "nunique"})
```

[df] Statistiques et mathématiques

DESCRIPTION : Calcule des statistiques descriptives et des opérations mathématiques
 POURQUOI : Analyser et résumer les données numériques.
 CONTEXTE : Analyse exploratoire, rapports statistiques, feature engineering.
 FONCTIONS PRINCIPALES :

df.sum()	Somme par colonne
df.sum(axis=1)	Somme par ligne
df.mean()	Moyenne
df.median()	Médiane
df.mode()	Mode (valeur la plus fréquente)
df.min()	Minimum
df.max()	Maximum
df.std()	Écart-type
df.var()	Variance
df.sem()	Erreur standard de la moyenne
df.skew()	Asymétrie (skewness)
df.kurt()	Kurtosis
df.abs()	Valeur absolue
df.round(2)	Arrondi à N décimales
df.clip(lower=0, upper=100)	Borne les valeurs
df.quantile(0.25)	1er quartile
df.quantile([0.25, 0.5, 0.75])	Quartiles multiples
df.cumsum()	Somme cumulée
df.cumprod()	Produit cumulé
df.cummax()	Maximum cumulé
df.cummin()	Minimum cumulé
df.diff(periods=1)	Différence avec la ligne N-1
df.pct_change()	Variation en %
df.rank()	Rang de chaque valeur
df.rank(method="dense")	Rang dense (sans saut)
df.rank(pct=True)	Rang en percentile
df.corr()	Matrice de corrélation
df.corr(method="spearman")	Corrélation de Spearman
df.cov()	Matrice de covariance
df["a"].corr(df["b"])	Corrélation entre deux Series
df["a"].cov(df["b"])	Covariance entre deux Series
df.dot(autre)	Produit matriciel

df.prod()	Produit de toutes les valeurs
df.idxmax()	Index de la valeur max
df.idxmin()	Index de la valeur min
df.nlargest(n, "col")	N plus grandes valeurs
df.nsmallest(n, "col")	N plus petites valeurs
df.count()	Nombre de valeurs non-nulles
df.eval("c = a + b", inplace=True)	Calcul d'expression
df.add(outre, fill_value=0)	Addition avec fill
df.sub(outre, fill_value=0)	Soustraction avec fill
df.mul(outre, fill_value=1)	Multiplication avec fill
df.div(outre, fill_value=1)	Division avec fill
df.pow(2)	Élévation à la puissance
df.mod(outre)	Modulo
df.floordiv(outre)	Division entière
df.truediv(outre)	Division réelle
df.all()	Vrai si toutes valeurs vraies
df.any()	Vrai si au moins une valeur vraie

EXEMPLES :

```
df[["a","b"]].corr()
df.nlargest(10, "montant")[["client","montant"]]
```

[df] Tri – sort_values, sort_index

DESCRIPTION : Trie les lignes ou colonnes d'un DataFrame

POURQUOI : Ordonner les données pour l'affichage, les analyses ou les jointures.

CONTEXTE : Rapports classés, préparation de jointures, classements.

FONCTIONS PRINCIPALES :

df.sort_values("col")	Tri croissant par colonne
df.sort_values("col", ascending=False)	Tri décroissant
df.sort_values(["col1","col2"])	Tri multi-colonnes
df.sort_values("col", na_position="first")	NaN en premier
df.sort_values("col", na_position="last")	NaN en dernier
df.sort_values("col", key=lambda x: x.str.lower())	Tri avec transformation
df.sort_index()	Tri par index croissant
df.sort_index(ascending=False)	Tri par index décroissant
df.sort_index(axis=1)	Tri des colonnes par nom
df.nlargest(n, "col")	N plus grandes valeurs (trié)
df.nsmallest(n, "col")	N plus petites valeurs (trié)
df["col"].argsort()	Indices qui trieraient la colonne

EXEMPLES :

```
df.sort_values(["région","ca"], ascending=[True, False])
df.nlargest(5, "chiffre_affaires")
```

[df] Regroupement et agrégation – groupby

DESCRIPTION : Regroupe les lignes par valeurs et applique des fonctions d'agrégation

POURQUOI : Calculer des statistiques par groupe (par région, par catégorie...).

CONTEXTE : Tableaux de synthèse, rapports d'agrégation, analyses par segment.

FONCTIONS PRINCIPALES :

df.groupby("col")	Groupe par une colonne
df.groupby(["col1","col2"])	Groupe par plusieurs colonnes
df.groupby("col", as_index=False)	Résultat sans index groupé
df.groupby("col", dropna=False)	Inclut les groupes NaN
df.groupby("col").sum()	Somme par groupe
df.groupby("col").mean()	Moyenne par groupe
df.groupby("col").median()	Médiane par groupe
df.groupby("col").min()	Minimum par groupe
df.groupby("col").max()	Maximum par groupe
df.groupby("col").count()	Comptage par groupe
df.groupby("col").std()	Écart-type par groupe
df.groupby("col").var()	Variance par groupe
df.groupby("col").first()	Première valeur par groupe
df.groupby("col").last()	Dernière valeur par groupe
df.groupby("col").nunique()	Valeurs uniques par groupe
df.groupby("col").size()	Taille de chaque groupe
df.groupby("col").agg("sum")	Agrégation simple
df.groupby("col").agg(["sum","mean"])	Plusieurs agrégations
df.groupby("col").agg({"a":"sum","b":"mean"})	Agrégation différente par col
df.groupby("col").agg(total=("val","sum"))	Avec renommage (NamedAgg)
df.groupby("col").apply(fonction)	Fonction personnalisée
df.groupby("col").transform("mean")	Transforme (même shape que df)
df.groupby("col").filter(lambda x: len(x)>5)	Filtre des groupes entiers
df.groupby("col").cumsum()	Somme cumulée par groupe

<code>df.groupby("col").rank()</code>	Rang au sein du groupe
<code>df.groupby("col").diff()</code>	Différence au sein du groupe
<code>df.groupby("col").pct_change()</code>	Variation % au sein du groupe
<code>df.groupby("col").head(3)</code>	3 premières lignes par groupe
<code>df.groupby("col").tail(3)</code>	3 dernières lignes par groupe
<code>df.groupby("col").nth(0)</code>	N-ième ligne par groupe
<code>df.groupby("col").ngroups</code>	Nombre de groupes
<code>df.groupby("col").groups</code>	Dictionnaire des groupes
<code>df.groupby("col").get_group("valeur")</code>	Obtient un groupe spécifique
<code>pd.NamedAgg(column="val", aggfunc="sum")</code>	Agrégation nommée
<code>pd.Grouper(freq="M")</code>	Grouper temporel

EXEMPLES :

```
df.groupby("région", as_index=False).agg(
    ca_total=("montant", "sum"),
    nb_clients=("client_id", "nunique"),
    panier_moyen=("montant", "mean")
)
```

[df] Fusion et jointure – merge, join, concat

DESCRIPTION : Combine plusieurs DataFrames entre eux

POURQUOI : Enrichir les données, recouper des tables, assembler des datasets.

CONTEXTE : Jointures de tables, assemblage de datasets, ETL.

FONCTIONS PRINCIPALES :

<code>pd.merge(df1, df2, on="col")</code>	Jointure sur une colonne
<code>pd.merge(df1, df2, on=["c1", "c2"])</code>	Jointure multi-colonnes
<code>pd.merge(df1, df2, how="inner")</code>	Jointure interne (défaut)
<code>pd.merge(df1, df2, how="left")</code>	Jointure gauche
<code>pd.merge(df1, df2, how="right")</code>	Jointure droite
<code>pd.merge(df1, df2, how="outer")</code>	Jointure externe complète
<code>pd.merge(df1, df2, how="cross")</code>	Produit cartésien
<code>pd.merge(df1, df2, left_on="a", right_on="b")</code>	Colonnes de jointure différentes
<code>pd.merge(df1, df2, left_index=True, ...)</code>	Jointure sur les index
<code>pd.merge(df1, df2, suffixes=("_g", "_d"))</code>	Suffixes en cas de doublon
<code>pd.merge(df1, df2, indicator=True)</code>	Colonne _merge pour diagnostic
<code>pd.merge_asof(df1, df2, on="date")</code>	Jointure approximative (timeseries)
<code>pd.merge_ordered(df1, df2, on="date")</code>	Jointure avec tri
<code>df1.join(df2, how="left")</code>	Jointure sur les index
<code>df1.join(df2, on="col")</code>	Jointure df1.col = df2.index
<code>pd.concat([df1, df2])</code>	Empile verticalement (axis=0)
<code>pd.concat([df1, df2], axis=1)</code>	Empile horizontalement
<code>pd.concat([df1, df2], ignore_index=True)</code>	Réinitialise l'index
<code>pd.concat([df1, df2], keys=["a", "b"])</code>	Ajoute un niveau d'index
<code>pd.concat([df1, df2], join="inner")</code>	Garde seulement les colonnes communes
<code>df1.combine(df2, func)</code>	Combine avec une fonction
<code>df1.combine_first(df2)</code>	df1 prioritaire, df2 pour combler
<code>df1.update(df2)</code>	Met à jour df1 avec df2 (in-place)
<code>df1.align(df2)</code>	Aligne deux DataFrames sur les index

EXEMPLES :

```
df = pd.merge(clients, commandes, on="client_id", how="left")
df = pd.concat([df_2022, df_2023, df_2024], ignore_index=True)
df = pd.merge_asof(prix, événements, on="date", direction="backward")
```

[df] Pivotement et restructuration

DESCRIPTION : Restructure la forme du DataFrame (long/large, pivot, melt)

POURQUOI : Passer d'un format long à large ou inversement selon les besoins.

CONTEXTE : Préparation pour visualisation, tableaux croisés, reshaping.

FONCTIONS PRINCIPALES :

<code>df.pivot(index="i", columns="c", values="v")</code>	Pivote sans agrégation
<code>df.pivot_table(index="i", columns="c", values="v", aggfunc="sum")</code>	Pivote avec agrégation
<code>pd.pivot_table(df, values="v", index="i", columns="c", aggfunc="mean")</code>	Fonction top-level
<code>df.melt(id_vars=["id"])</code>	Long format (toutes colonnes)
<code>df.melt(id_vars=["id"], value_vars=["a", "b"])</code>	Long format (colonnes choisies)
<code>df.melt(var_name="variable", value_name="val")</code>	Noms personnalisés
<code>pd.melt(df, id_vars=["id"])</code>	Fonction top-level
<code>pd.wide_to_long(df, stubnames=["val"], i="id", j="année")</code>	Large → long avec suffixes
<code>df.stack()</code>	Colonnes → index (long)
<code>df.unstack()</code>	Index → colonnes (large)
<code>df.stack(level=-1)</code>	Dernier niveau de colonnes

df.unstack(level=0)	Premier niveau d'index
pd.crosstab(df["a"], df["b"])	Tableau croisé de fréquences
pd.crosstab(df["a"], df["b"], values=df["v"], aggfunc="sum")	Tableau croisé avec valeurs
df.explode("col")	Explose une liste en lignes
df.squeeze()	Réduit les dimensions inutiles
df.transpose()	Transpose lignes/colonnes
df.T	Raccourci transpose

EXEMPLES :

```

tableau = df.pivot_table(index="région", columns="catégorie",
                          values="montant", aggfunc="sum", fill_value=0)
df_long = df.melt(id_vars=["date", "région"], var_name="produit", value_name="ventes")

```

[df] Fenêtres glissantes – rolling, expanding, ewm

DESCRIPTION : Calcule des statistiques sur des fenêtres glissantes ou cumulées
 POURQUOI : Lissage de séries temporelles, moyennes mobiles, tendances.
 CONTEXTE : Finance, séries temporelles, détection de tendances.

FONCTIONS PRINCIPALES :

df.rolling(N)	Fenêtre glissante de N périodes
df.rolling(N).mean()	Moyenne mobile
df.rolling(N).sum()	Somme glissante
df.rolling(N).std()	Écart-type glissant
df.rolling(N).min()	Minimum glissant
df.rolling(N).max()	Maximum glissant
df.rolling(N).apply(func)	Fonction personnalisée glissante
df.rolling(N, min_periods=1)	Périodes minimum requises
df.rolling(N, center=True)	Fenêtre centrée
df.rolling("7D")	Fenêtre glissante temporelle
df.expanding()	Fenêtre croissante depuis début
df.expanding().mean()	Moyenne cumulative
df.expanding().sum()	Somme cumulative
df.expanding(min_periods=10)	Minimum de périodes
df.ewm(span=N)	Lissage exponentiel
df.ewm(span=N).mean()	Moyenne mobile exponentielle
df.ewm(alpha=0.3)	Lissage avec alpha
df.ewm(halflife="7D")	Demi-vie temporelle
df.ewm(com=N)	Centre de masse

EXEMPLES :

```

df["ma_7j"] = df["close"].rolling(7).mean()
df["ema_30"] = df["close"].ewm(span=30).mean()
df["vol_30j"] = df["rendement"].rolling(30).std() * (252**0.5)

```

[df] Application de fonctions – apply, map, transform, pipe

DESCRIPTION : Applique des fonctions personnalisées sur les données
 POURQUOI : Transformer les données avec une logique métier personnalisée.
 CONTEXTE : Feature engineering, transformations complexes, normalisation.

FONCTIONS PRINCIPALES :

df.apply(func)	Applique func sur chaque colonne
df.apply(func, axis=1)	Applique func sur chaque ligne
df.apply(func, result_type="expand")	Résultat expansé en colonnes
df["col"].apply(func)	Applique func sur une Series
df["col"].map(func)	Mappe element-par-element
df["col"].map({"a": 1, "b": 2})	Mappe via un dictionnaire
df.map(func)	Applique func sur chaque cellule
df.applymap(func)	Identique à map (déprécié)
df.transform(func)	Transforme, retourne même shape
df.transform(["mean", "std"])	Plusieurs transformations
df.groupby("col").transform("mean")	Transforme par groupe
df.agg(func)	Agrège sur l'axe
df.agg(["sum", "mean"])	Plusieurs agrégations
df.agg({"col1": "sum", "col2": "mean"})	Agrégation par colonne
df.pipe(func)	Applique func au DataFrame entier
df.pipe(func, arg=val)	Avec arguments supplémentaires
pd.eval("df['c'] = df['a'] + df['b']")	Évalue une expression globale

EXEMPLES :

```

df["score_normalisé"] = df["score"].apply(lambda x: (x - x.mean()) / x.std())
df.apply(lambda row: row["a"] * row["b"] if row["flag"] else 0, axis=1)
df.pipe(nettoyer).pipe(enrichir).pipe(normaliser)

```

[df] Catégories – astype("category"), cat.*

DESCRIPTION : Gère les données catégorielles pour optimiser mémoire et performance

POURQUOI : Les colonnes catégorielles occupent moins de mémoire et permettent un tri ordonné et des opérations groupby plus rapides.

CONTEXTE : Variables qualitatives, encodage, optimisation mémoire.

FONCTIONS PRINCIPALES :

<code>df["col"].astype("category")</code>	Convertit en catégorielle
<code>pd.Categorical(["a","b","a"])</code>	Crée une Series catégorielle
<code>pd.CategoricalDtype(categories=["a","b"], ordered=True)</code>	Dtype ordonné
<code>df["col"].cat.categories</code>	Liste des catégories
<code>df["col"].cat.codes</code>	Codes numériques des catégories
<code>df["col"].cat.ordered</code>	Vrai si ordonné
<code>df["col"].cat.add_categories(["c"])</code>	Ajoute des catégories
<code>df["col"].cat.remove_categories(["a"])</code>	Supprime des catégories
<code>df["col"].cat.remove_unused_categories()</code>	Supprime les catégories vides
<code>df["col"].cat.set_categories(["a","b"])</code>	Redéfinit les catégories
<code>df["col"].cat.rename_categories({"a":"A"})</code>	Renomme les catégories
<code>df["col"].cat.reorder_categories(liste)</code>	Réordonne les catégories
<code>df["col"].cat.as_ordered()</code>	Rend la colonne ordonnée
<code>df["col"].cat.as_unordered()</code>	Rend la colonne non-ordonnée
<code>pd.get_dummies(df["col"])</code>	One-hot encoding
<code>pd.get_dummies(df, drop_first=True)</code>	One-hot sans première catégorie
<code>pd.from_dummies(df_dummies)</code>	Inverse du one-hot encoding
<code>pd.factorize(df["col"])</code>	Encodage numérique + index
<code>pd.cut(df["col"], bins=5)</code>	Discrétisation en intervalles égaux
<code>pd.cut(df["col"], bins=[0,10,50,100])</code>	Discrétisation avec bornes
<code>pd.cut(df["col"], bins=5, labels=["TBas","Bas","Moy","Haut","THaut"])</code>	
<code>pd.qcut(df["col"], q=4)</code>	Discrétisation par quantiles
<code>pd.qcut(df["col"], q=4, labels=["Q1","Q2","Q3","Q4"])</code>	

EXEMPLES :

```
df["taille"] = pd.Categorical(df["taille"], categories=["S","M","L","XL"], ordered=True)
dummies = pd.get_dummies(df[["sexe","région"]], drop_first=True)
df["tranche_age"] = pd.cut(df["age"], bins=[0,18,35,60,100],
                           labels=["mineur","jeune","adulte","senior"])
```

[df] Index et MultiIndex

DESCRIPTION : Manipule les index simples et hiérarchiques

POURQUOI : L'index est la clé d'accès aux données et des jointures efficaces.

CONTEXTE : Données hiérarchiques, séries temporelles, jointures par index.

FONCTIONS PRINCIPALES :

<code>df.set_index("col")</code>	Définit une colonne comme index
<code>df.set_index(["col1","col2"])</code>	Index MultiIndex
<code>df.reset_index()</code>	Remet l'index en colonne
<code>df.reset_index(drop=True)</code>	Supprime l'index
<code>df.index.rename("nom")</code>	Renomme l'index
<code>df.reindex(nouvel_index)</code>	Réindexe (NaN si manquant)
<code>df.reindex(méthode="ffill")</code>	Remplit les valeurs manquantes
<code>df.sort_index()</code>	Trie par index
<code>df.index.is_unique</code>	Vrai si l'index est unique
<code>df.index.is_monotonic_increasing</code>	Vrai si index croissant
<code>df.index.duplicated()</code>	Masque des index dupliqués
<code>pd.MultiIndex.from_tuples(liste)</code>	MultiIndex depuis tuples
<code>pd.MultiIndex.from_product([L1,L2])</code>	MultiIndex produit cartésien
<code>pd.MultiIndex.from_frame(df)</code>	MultiIndex depuis DataFrame
<code>df.xs("val", level=0)</code>	Coupe transversale niveau 0
<code>df.swaplevel(0, 1)</code>	Échange deux niveaux d'index
<code>df.droplevel(0)</code>	Supprime un niveau d'index
<code>df.unstack(level=0)</code>	Transforme niveau index en colonnes
<code>df.stack()</code>	Transforme colonnes en index

EXEMPLES :

```
df = df.set_index(["année","mois"]).sort_index()
df.xs(2024, level="année")
```

[pd] Fonctions utilitaires top-level

DESCRIPTION : Fonctions pandas accessibles directement via `pd.*`

POURQUOI : Opérations générales non liées à un DataFrame spécifique.

CONTEXTE : Manipulation de données, conversions, utilitaires.

FONCTIONS PRINCIPALES :

<code>pd.to_numeric(série, errors="coerce")</code>	Convertit en numérique
<code>pd.to_datetime(série)</code>	Convertit en datetime

pd.to_timedelta(série)	Convertit en timedelta
pd.to_pickle(obj, "f.pkl")	Sérialise un objet
pd.unique(série)	Valeurs uniques (plus rapide)
pd.isna(valeur)	Teste si NaN / NaT / None
pd.notna(valeur)	Teste si non-nul
pd.isnull(valeur)	Identique à isna
pd.notnull(valeur)	Identique à notna
pd.concat(liste_df)	Concatène des DataFrames
pd.merge(df1, df2)	Fusionne deux DataFrames
pd.pivot(df)	Pivote un DataFrame
pd.pivot_table(df)	Tableau croisé dynamique
pd.melt(df)	Format long
pd.crosstab(a, b)	Tableau croisé de fréquences
pd.cut(série, bins)	Discrétisation par intervalles
pd.qcut(série, q)	Discrétisation par quantiles
pd.factorize(série)	Encodage numérique
pd.get_dummies(df)	One-hot encoding
pd.from_dummies(df)	Inverse one-hot
pd.json_normalize(data)	Normalise un JSON imbriqué
pd.lreshape(df, groups)	Reshape format long
pd.wide_to_long(df, stubnames, i, j)	Large → long
pd.eval(expression)	Évalue une expression string
pd.set_option("display.max_rows", 100)	Configure l'affichage
pd.get_option("display.max_rows")	Lit une option
pd.reset_option("display.max_rows")	Remet l'option par défaut
pd.describe_option("display.max_rows")	Décrit une option
pd.option_context("display.max_rows", 50)	Contexte d'option temporaire
pd.options.display.max_rows = 100	Accès direct aux options
pd.show_versions()	Affiche les versions installées
pd.infer_freq(index)	Infère la fréquence d'un index
pd.offsets.Day(1)	Offset d'un jour
pd.offsets.MonthEnd()	Fin de mois
pd.testing.assert_frame_equal(df1, df2)	Teste l'égalité (tests unitaires)
pd.testing.assert_series_equal(s1, s2)	Teste l'égalité de Series
pd.array(data, dtype)	Crée un ExtensionArray

EXEMPLES :

```

pd.set_option("display.max_columns", None)
with pd.option_context("display.float_format", "{:.2f}".format):
    print(df)
df["id"] = pd.to_numeric(df["id"], errors="coerce")
data = pd.json_normalize(json_data, record_path="items", meta=["date", "source"])

```